06
Scalable data structures and code in Scala

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

Scala has two key features

- mixes OOP and FP
- intrinsic scalability of concepts

This lesson will study mechanisms, libraries and interplay

1. how to write good functional data structures
2. the scala collection framework
3. nesting of constructs, and consequences

Outline

1. Developing functional data structures
2. Scala collections
3. On (im/)mutability
4. Structuring complex programs
   - Nesting
   - Packages and imports
   - Visibility modifiers

Recapping FP and OOP in Scala (so far)

FP in Scala

- function types/literals, higher-order functions, methods, genericity
- pattern matching (and absense of mutability)
- objects/modules, module types/ADT, product types and sum types
- lazy evaluation

OOP in Scala

- key elements as in Java, but purer
- inheritance, delegation, interfaces (traits), possibly mutability
- class parameters, auxiliary constructors
- object as singleton and/or collector of ex-static properties
- uniform access (every operation is a method call)

OOP/FP combination in Scala

Shape of Scala

- classes/objects as main constructs
- methods filled in functional style
- very advanced type system (generics, variance, dependent types, functors, . . .)
- possibly use advanced FP patterns (typically in libraries)

The three levels of Scala language/framework

- BOTTOM: Java/JVM (efficiency, interoperability . . . , still with JS/Native support)
- MIDDLE: pure, modern OOP (a better Java: everything is an object, everything is a method call, promote immutability)
- TOP: additional sugar for FP, additional sugar for flexibility/conciseness/DSL support

Working with data structures in Scala

Importance of collections

- already known as key in Java
- all applications manage sets/iterations of objects somehow
- in Scala they are even more important, conceptually, since they well integrate with lazy lists, optionals (and *monads*)
- exercise development of collections will give us hints on how to write structures (data+behaviour) in general

Building a functional data-type: review

Requirements

- seeking for immutability, when possible (mostly depend on performance)
- "non-bug exceptions" handled with optionals
- supporting declarative map-filter-foreach operations

Three approaches

- fully-functional: seen in previous lessons
  - separately define type (enum), constructors and FP-style operations
  - type definition comes opaque in the module with ADTs
- fully-OO
  - as in Java: define type including OO-style operations signature, and one class per different implementation
- Scala's OO/FP mixture
  - define type including OO-style operations signature, constructors, and then a single algorithm-oriented implementation

An example: cons/nil linked lists

- the simplest construction, yet the most used one in FP (due to easy matching)
- will show OO/FP solutions

The trick of right associative construction/extraction

```scala
object RightAssociativeConstruction extends App:

  enum List[A]: // a type with two implementations and methods
    case ::(h: A, t: List[A]) // just a different name for Cons
    case Nil()
    def ::(h: A): List[A] = List.::(h, this)
    def isEmpty: Boolean = this == Nil() // another method

  object List:
    def head(l: List[Int]): Option[Int] = l match
      case h :: _ => Some(h) // same as ::(h, _) => Some(h)
      case _ => None

  import List.*

  val l: List[Int] = ::(10, ::(20, ::(30, Nil()))) // using case ::(h, t)
  val l2: List[Int] = Nil().::(30).::(20).::(10)   // using def ::
  val l3: List[Int] = 10 :: 20 :: 30 :: Nil()      // using infix notation
  // recall that operators ending with : have right-associativity

  println(s"$l, $l2, $l3, ${l.isEmpty}")
  println(head(l)) // FP-like call
  println(l.isEmpty) // OO-like call
```

Design of OOP/FP idiomatic Scala approach

Mixed OO/FP

- a `enum List` with OO-style method signatures (`head`, `tail`, `append`, `map`)
- "`case class Cons`" and "`case class Nil`" as data structures (or algebraic data types)
- factories or FP-style operations into `List` companion object

The "::" trick

- When seen as a pattern, an infix notation `p op q` is equivalent to `op(p, q)`
- names ending with "`:`" are handled as right associative
- use name `::` instead of `Cons` for ease of matching
- use name `::` coherently for prepend operation

Type: definition and implementation

```scala
// List as a pure interface
enum List[A]:
  case ::(h: A, t: List[A])
  case Nil()
  def ::(h: A): List[A] = List.::(h, this)

  def head: Option[A] = this match
    case h :: t => Some(h)  // pattern for scala.Option
    case _ => None          // pattern for scala.Option

  def tail: Option[List[A]] = this match
    case h :: t => Some(t)
    case _ => None

  def append(list: List[A]): List[A] = this match
    case h :: t => h :: t.append(list)
    case _ => list

  def foreach(consumer: A => Unit): Unit = this match
    case h :: t => consumer(h); t.foreach(consumer)
    case _ =>

  def get(pos: Int): Option[A] = this match
    case h :: t if pos == 0 => Some(h)
    case h :: t if pos > 0 => t.get(pos - 1)
    case _ => None

  def filter(predicate: A => Boolean): List[A] = this match
    case h :: t if predicate(h) => h :: t.filter(predicate)
    case _ :: t => t.filter(predicate)
    case _ => Nil()

  def map[B](fun: A => B): List[B] = this match
    case h :: t => fun(h) :: t.map(fun)
    case _ => Nil()
```

Object and few tests

```scala
object List:

  def apply[A](elems: A*): List[A] =
    var list: List[A] = Nil()
    for e <- elems.reverse do list = e :: list
    list

  def of[A](elem: A, n: Int): List[A] =
    if n == 0 then Nil() else elem :: of(elem, n - 1)

object Test extends App:

  import List.*

  println(List(10, 20, 30, 40))
  val l = 10 :: 20 :: 30 :: 40 :: Nil() // same as above
  println(l.head) // Some(10)
  println(l.tail) // Some(...20,30,40...)
  println(l.append(l)) // 10,20,30,40,10,20,30,40
  println(l append l) // 10,20,30,40,10,20,30,40
  println(l get 2) // Some(30)
  println(of("a", 10)) // a,a,a,..,a
  println(l filter (_ <= 20) map ("a" + _)) // a10, a20
  println(l.filter(_ <= 20).map("a" + _)) // a10, a20
  println(for i <- l do println(i)) // uses foreach

  println(l.append(l) == l) // false, note intrinsic immutability
```

Folding

```scala
enum List[A]:
  case ::(h: A, t: List[A])
  case Nil()
  def ::(h: A): List[A] = List.::(h, this)

  // foldLeft capturing left-to-right accumulation, typical of reductions
  @tailrec final def foldLeft[B](init: B)(f: (B, A) => B): B = this match
    case h :: t => t.foldLeft(f(init, h))(f)
    case _ => init

  def size(init: Int = 0): Int = foldLeft(init)((acc, _) => acc + 1)

  override def toString: String = foldLeft("|")((acc, h) => acc + h.toString + "|")

  // foldRight capturing right-to-left construction
  def foldRight[B](init: B)(op: (A, B) => B): B = this match
    case h :: t => op(h, t.foldRight(init)(op))
    case _ => init

  def map[B](f: A => B): List[B] = foldRight(Nil())(f(_) :: _)

  def concat(l: List[A]): List[A] = foldRight(l)(_ :: _)

  // right-to-left construction with tail recursion, needs reverse
  def reverse: List[A] = foldLeft(Nil[A]())((acc, h) => h :: acc)

  def mapTail[B](f: A => B): List[B] =
    foldLeft(Nil[B]())((acc, h) => f(h) :: acc).reverse
```

On the mixed OO/FP solution

Internal features

- clear separation: contract, construction/extraction, algorithms
- still possible to implement some algorithms in the OO style
- many algorithms seem similar, might apply refactoring patterns

External features

- the OO calling style supports method chaining like for Java 8+ streams
- very flexible construction/use operations
- `foreach`, `map` supports for-do comprehension

⇒ `flatMap` would support multiple variables for-yield comprehension

⇒ `filter` would support conditionals in for-comprehension

What about encapsulation?

- FP promotes type classes as a way to encapsulate behaviour rather than data

Mutabile data structures: idiomatic mutable list

Internal features

- essentially a class holding optional head and tail
- head and tail are mutable (if head is empty, tail is empty)
- operations to add elements could yield unit or (fluent style) this

Structure

- (can't use algebraic data types)
- standard trait + object with private implementation
- note implementation is trickier: immutability is definitely easier!
- note mutable operations might have same signature as in the immutable case, they just return `this`!

⇒ so in principle, mutable and immutable collections might have same interface

MutableList trait and use

```scala
trait MutableList[A]:
  def head: Option[A]
  def tail: Option[MutableList[A]]
  def prepend(h: A): MutableList[A]
  def append(l: MutableList[A]): MutableList[A]
  def map[B](f: A => B): MutableList[B]
  def toList: List[A]
```

```scala
object TestMutableList extends App:

  import MutableList.*

  val l: MutableList[Int] = MutableList()
  l.prepend(40).prepend(30).prepend(20).prepend(10)
  println(l.toList) // 10,20,30,40
  println(l.head) // Some(10)
  println(l.tail.get.toList) // 20,30,40
  val l2: MutableList[Int] = MutableList()
  l2.prepend(60).prepend(50)
  l.append(l2) // what would happen with l.append(l) ???
  println(l.toList) // 10,20,30,40,50,60
  println(l.map(_ + 1).toList) // 11,21,31,41,51,61
```

MutableList companion and implementation

```scala
object MutableList:

  def apply[A](): MutableList[A] = MutableListImpl(None)

  private case class MutableListImpl[A](
      var ht: Option[(A, MutableList[A])]) extends MutableList[A]:

    override def head: Option[A] = ht.map(_._1)

    override def tail: Option[MutableList[A]] = ht.map(_._2)

    override def prepend(h: A): MutableList[A] =
      ht = Some((h, MutableListImpl(ht)))
      this

    override def append(l: MutableList[A]): MutableList[A] =
      ht match
        case None if l.head.isEmpty => {} // [],[] --> []
        case None => ht = Some((l.head.get, l.tail.get)) // [],[H|T] --> [H|T]
        case Some((h, t)) if t.head.isEmpty => ht = Some((h, l)) // [H],L --> [H|L]
        case Some((h, t)) => ht = Some((h, t.append(l))) // [H|T],L --> [H|T.append(L)]
      this

    override def map[B](f: A => B): MutableList[B] =
      MutableListImpl(ht.map((h, t) => (f(h), t.map(f))))

    override def toList: List[A] = ht match
      case None => List.Nil()
      case Some((h, t)) => List.::(h, t.toList)
```

On (im/)mutability and performance considerations

Brief recap

- mutable structures are more efficient if they are frequently changed
- clever immutable structures still can have e.g. logarithmic-time-space operations to updated structures: they can largely reuse portions of the original data structure

$\Rightarrow$ think at immutable red-black trees

Other considerations

- immutable collections are intrinsically thread-safe
- immutable collections can be shared
- immutable collections can be easily used to extract information (e.g. by pipelines)
- side-effects in mutable structures might be tricky

Outline

1. Developing functional data structures
2. Scala collections
3. On (im/)mutability
4. Structuring complex programs
   - Nesting
   - Packages and imports
   - Visibility modifiers

Scala collection framework

Key facts

- main reference: `https://docs.scala-lang.org/scala3/book/collections-classes.html`
- whole API: https://scala-lang.org/api/3.5.1/
- declared goals: easy, concise, safe, fast, parallel, universal
- main interfaces in package `scala.collection`, implementations are nested
- support mutability and immutability (each collection has both), packages `scala.collection.immutable` and `scala.collection.mutable`
  - both interfaces and implementations
- support various abstractions: `Set`, `Seq`, `Map`
  - roughly similar to Java's sets, lists and maps
- support abstractions to somewhat address optimisation
  - sequences are either linear or indexed, lazylists are linear sequences
- support many many algorithms in each abstraction
  - even the root interface has ˜ 100 methods
- support various data structures (˜ 80 classes)
- provides simple access (via aliases) for default implementations
- `scala.Option` is somewhat part of the collection framework

Architecture

IterableOnce and Iterable

- `trait IterableOnce` basically includes only abstract `iterator` method
- `trait Iterable` adds template methods for various basic operations (`fold`, `flatMap`)

Set, Seq, and Map

- `trait Set` models collections without duplicate elements
- `trait Seq` models indexed collections
  - `LinearSeq` optimises access to `head` and `tail`
  - `IndexedSeq` optimises `apply` and `length`
- `trait Map` models key-value pairs

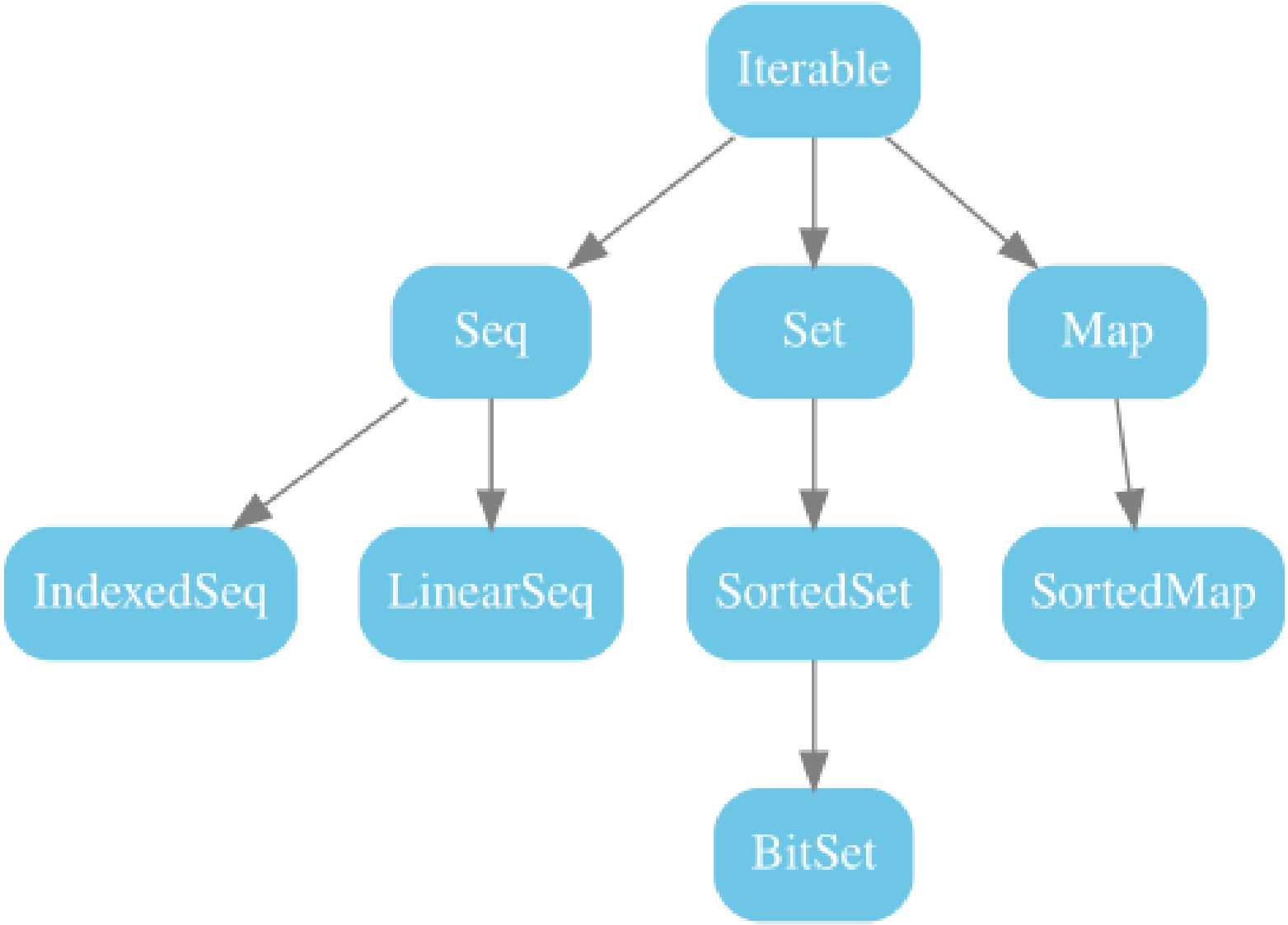
Common implementations

- `LinearSeq`: `immutable.List`, `immutable.LazyList`, `mutable.ListBuffer`
- `IndexedSeq`: `scala.Array`, `mutable.ArrayBuffer`, `immutable.Vector`
- `Set`: `im/mutable.HashSet`
- `Map`: `im/mutable.HashMap`

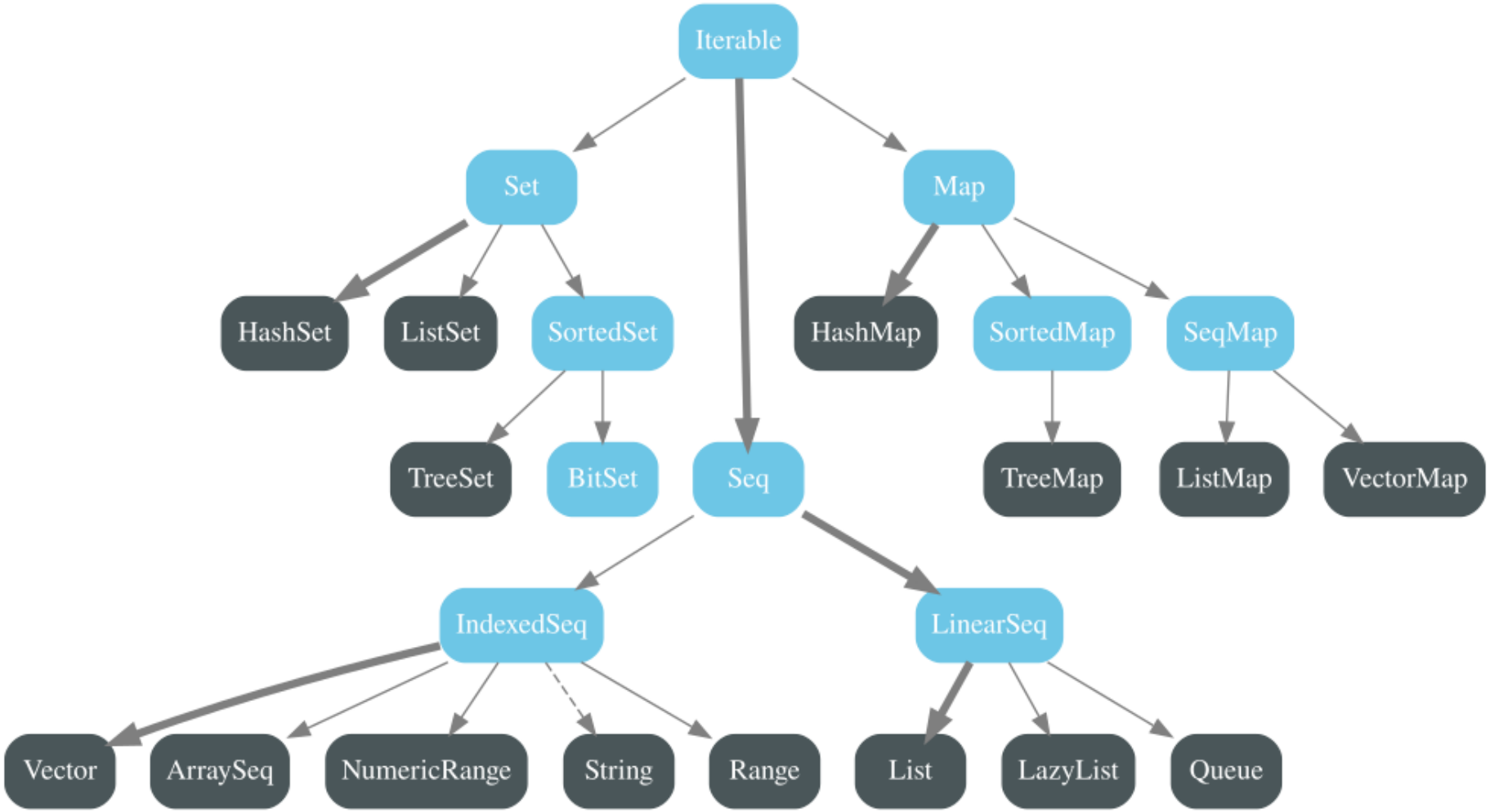
Creating a collection by Iterable

```
object WorkingWithIterable extends App:

  class Fibonacci(n: Int) extends Iterable[Int]:
    override def iterator: Iterator[Int] = new Iterator[Int]():
      var counter = 0
      var prev = (0, 1)
      override def next(): Int =
        prev = (prev._2, prev._1 + prev._2)
        counter = counter + 1
        prev._1
      override def hasNext: Boolean = counter <= n

  val fibs = Fibonacci(20)
  val fibsAsStrings = fibs.map(i => s"Next one is $i")
  val list = fibs.toList
```

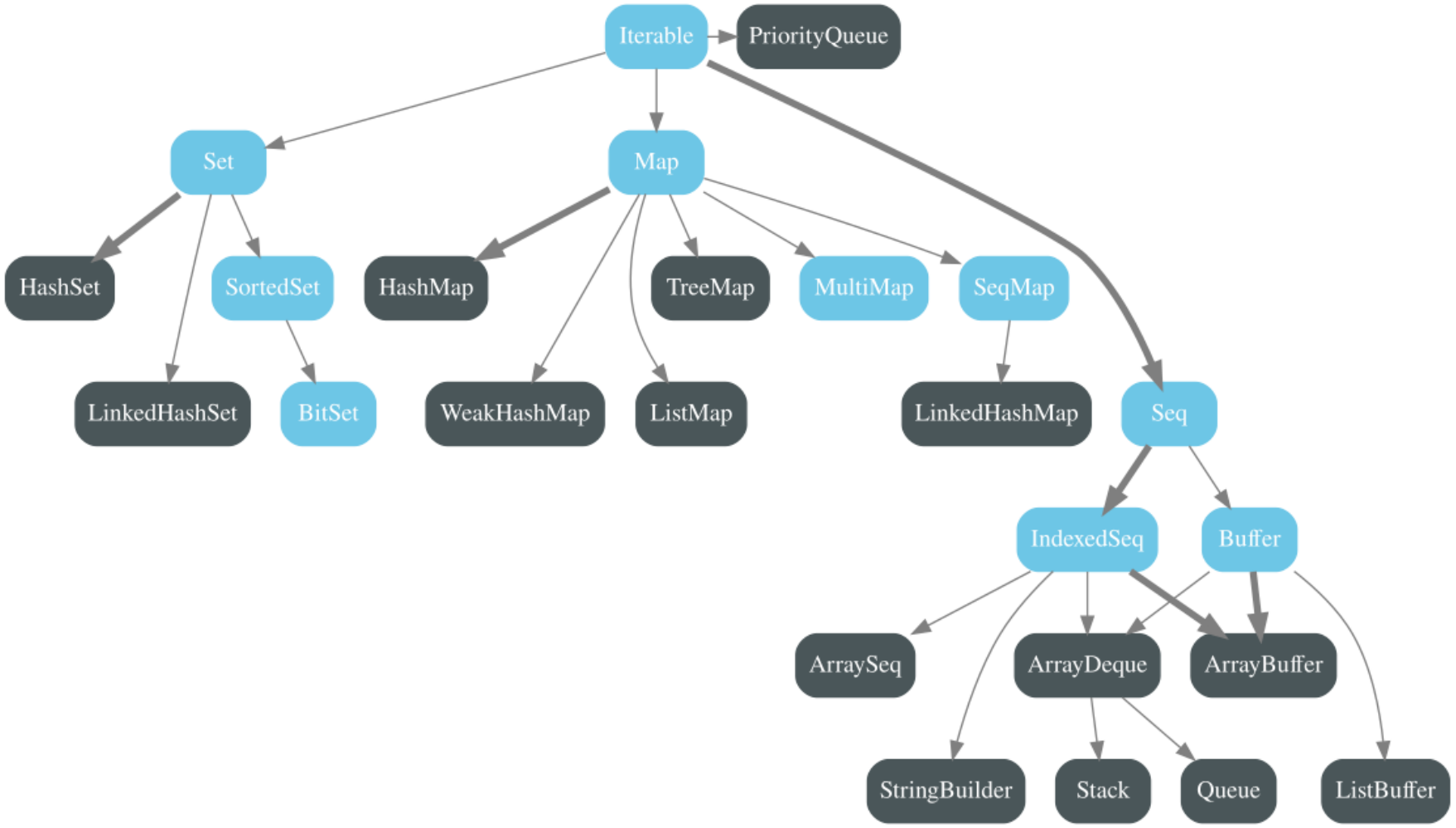
Interfaces


Iterable
Seq
Set
Map
IndexedSeq
LinearSeq
SortedSet
SortedMap
BitSet

Immutable collections



Mutable collections



First example: working with lists (linear sequences)

```scala
// list is an immutable linear sequence, optimising head and tail access
val l: List[Int] = List(10,20,30,40,50) // works without imports
println(l.getClass) // class scala.collection.immutable.$colon$colon
println(l.isInstanceOf[scala.collection.immutable.List[_]]) // true
println(l) // List(10,20,30,40,50)
println(l.reverse) // List(50, 40, 30, 20, 10)
println(l.zipWithIndex) // List((10,0), (20,1), (30,2), (40,3), (50,4))
println(l.splitAt(3)) // a tuple.. (List(10, 20, 30),List(40, 50))
println(l(2)) // 30
println(5 :: l) // List(5, 10, 20, 30, 40, 50)
println(l ::: l) // List(10, 20, 30, 40, 50, 10, 20, 30, 40, 50)
println(l :+ 51) // List(10, 20, 30, 40, 50, 51)
println((l :+ 11).sortWith(_<_)) // List(10, 11, 20, 30, 40, 50)
println(l collect {case i if i > 20 => i + 1})  // List(31, 41, 51)
println(List.range(0,10)) // List(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
println(List.iterate(0,5)(_+3)) // List(0, 3, 6, 9, 12)

// working with mutable lists: a buffer for lists
import scala.collection.mutable.ListBuffer
val fruits = ListBuffer[String]() // mutable buffer to list
fruits += "Apple"  // means: fruits.+=("Apple")
fruits += "Banana"
fruits += ("Strawberry", "Kiwi")
fruits -= "Apple"
println(fruits) // ListBuffer(Banana, Strawberry, Kiwi)
println(fruits.toList) // List(Banana, Strawberry, Kiwi)
```

Working with lazy lists (lazy linear sequences)

```scala
// list is an immutable linear sequence, optimising head and tail access
val s: LazyList[Int] = LazyList(10,20,30,40,50) // works without imports
println(s.getClass) // class scala.collection.immutable.LazyList
println(s.isInstanceOf[scala.collection.immutable.LazyList[_]]) // true
println(s) // LazyList(<not computed>)
println(s(2)) // 30
println(s) // LazyList(10, 20, 30, <not computed>)
val s2 = s.zipWithIndex
println(s2) // LazyList(<not computed>)
println(s2(2)) // 30
println(s2) // LazyList((10,0), (20,1), (30,2), <not computed>)
println(s.toList) // List(10, 20, 30, 40, 50)
println(s) // LazyList(10, 20, 30, 40, 50)
```

Working with vectors/arrays (indexed sequences)

```
val v: Vector[Int] = Vector(10,20,30,40,50)
println(v.getClass) // class scala.collection.immutable.Vector1
// it supports all operations of lists..
println(v.reverse) // Vector(50, 40, 30, 20, 10)
println(v(2)) // 30

// scala.Array is backened by Java arrays!
// Array is a mutable indexed sequence, with constant length
val a: Array[Int] = Array(10,20,30)
println(a) // [I@7f690630
println(a.getClass) // class [I, namely an int[] !!
// it supports all operations of lists..
println(a.reverse) // [I@6bf256fa
println(a.reverse.toList) // List(30, 20, 10)

import collection.*
// ArrayBuffer is the default (fully) mutable indexed sequence
val b: mutable.ArrayBuffer[Int] = mutable.ArrayBuffer[Int]()
println(b) // ArrayBuffer
println(b.getClass) // class scala.collection.mutable.ArrayBuffer
b.addAll(List(10,20,30,40))
println(b)  // ArrayBuffer(10, 20, 30, 40)
println(b.toArray) // [I@62043840
println(b.toVector) // to an immutable sequence
```

Working with sets

```
// an immutable set, i.e., no sequencing and no duplications
val s: Set[Int] = Set(10,20,30,40,50)
println(s.getClass) // class scala.collection.immutable.
   HashSet$HashTrieSet
println(s) // Set(10, 20, 50, 40, 30)
// it supports most operations of lists.. and also..
println(s + 60 + 70) // Set(10, 20, 60, 70, 50, 40, 30)
println(s & Set(10,30)) // intersection.. Set(10)
println(s.contains(10)) // containment of an element

import collection._
// mutable set, aliasing a HashSet
val b: mutable.Set[Int] = mutable.Set(10,20,30)
println(b.getClass) // class scala.collection.mutable.HashSet
println(b) // Set(30, 20, 10)
b += 40    // means b.+=(40)
b += 40
println(b) // Set(30, 20, 10, 40)
```

Working with maps

```scala
// an immutable map, i.e., a key-value store
val m: Map[Int,String] = Map(10 -> "a",20 -> "b",30 -> "c")
println(m.getClass) // class scala.collection.immutable.Map$Map3
println(m) // Map(10 -> a, 20 -> b, 30 -> c)
println(m.toVector) // Vector((10,a), (20,b), (30,c))
println(m.values)   // Iterable(a, b, c)
println(m.keys)     // Set(10, 20, 30)
println(m(10)) // "a"
println(m.map{ case (k,v) => (v,k+".")}) // Map(a -> 10., b -> 20., c -> 30.)
println(m.map{ case (k,v) => v+v}) // List(aa, bb, cc)

// a mutable map
import collection._
val mm: mutable.Map[Int,String] = mutable.Map[Int,String]()
println(mm.getClass) // class scala.collection.mutable.HashMap
println(mm) // Map()
mm += (10 -> "a")
mm += (20 -> "b")
mm += (30 -> "c")
println(mm) // Map(20 -> b, 10 -> a, 30 -> c)
```

Working with options

Many methods exist: it is idiomatic to chain calls and neglect handling cases

```scala
object UseOption extends App :

  val some: Option[Int] = Some(10)
  val none: Option[Int] = None

  println(some.isDefined) // true

  println(some.fold(-1)(_ + 1)) // 11
  println(none.fold(-1)(_ + 1)) // -1

  println(some map (_ + 1)) // Some(11)
  println(none map (_ + 1)) // None

  println(some filter (_ > 0)) // Some(10)
  println(none filter (_ > 0)) // None
  println(Some(-5) filter (_ > 0)) // None

  println(some.flatMap(x => if x > 0 then Some(1) else None)) // Some(1)
  println(some.flatMap(x => Some(1) filter (i => x > 0))) // Better than the above
  println(none.flatMap(x => Some(0) filter (i => x > 0))) // None

  val some2: Option[Int] = Some(20)
```

Aliases and constructors

Constructors

- each basic interface/trait has its own companion with an `apply` method taking a variable argument
- immutable `Iterable`, `Seq`, `LinearSeq` and all below create a `List` (`IndexedSeq` creates a `Vector`)
- mutable `Iterable`, `Seq`, `IndexedSeq` and all below create an `ArrayBuffer`
- immutable/mutable `Set` create a mutable/immutable `HashSet`
- immutable/mutable `Map` create a mutable/immutable `HashMap`

Aliases requiring no import

- `Iterable`, `Seq`, `IndexedSeq`, `Set`, `Map` and `List` alias the same version in `collection.immutable`
- e.g., without importing `scala.collection.immutable.Set`, type `Set` means `scala.collection.immutable.Set`
- will see how aliasing works with the `type` construct

Abstractions/classes

```scala
object Collections extends App:

  def print[A](coll: Iterable[A]) = println(s"$coll,${coll.getClass}")

  print(Seq(10,20,30,40,50)) // ::
  print(scala.collection.immutable.LinearSeq(10,20,30,40,50)) // ::
  print(IndexedSeq(10,20,30,40,50)) // Vector1
  print(Set(10,20,30,40,50)) // HashSet
  print(scala.collection.immutable.SortedSet(10,20,30,40,50)) // TreeSet
  print(Map(10 -> 11,20 -> 21,30 -> 31,40 -> 41))// HashMap$Map4

  print(Array(10,20,30,40)) //.. mutable.ArraySeq$ofInt

  print(scala.collection.mutable.Seq(10,20,30,40,50)) // ArrayBuffer
  print(scala.collection.mutable.IndexedSeq(10,20,30,40,50)) // ArrayBuffer
  print(scala.collection.mutable.Set(10,20,30,40,50)) // HashSet
  print(scala.collection.mutable.SortedSet(10,20,30,40,50)) // TreeSet
  print(scala.collection.mutable.Map(10 -> 11,20 -> 21,30 -> 31,40 -> 41)) //HashMap
```

Scala views

Features

- another implementation of an `Iterable`
- a view is a lazy and immutable representation of a collection
- usable as dynamic immutable wrapper

```scala
object Views extends App:

  val list = collection.mutable.ListBuffer(10, 20, 30, 40)
  val view = list.view

  println(view) // SeqView(<not computed>)
  println(view.map(_ + 1))  // SeqView(<not computed>)
  println(view.map(_ + 1).toList) // List(11, 21, 31, 41)

  list.addOne(50)
  println(list) // ListBuffer(10, 20, 30, 40, 50)
  println(view.toList)  // List(10, 20, 30, 40, 50)

  println(view) // SeqView(<not computed>)
  for i <- view do println(i)  // 10 20 30...
  println(view) // SeqView(<not computed>)
```

1-slide summary of Scala collections

Architecture

- traits in `scala.collection`, impl in `scala.collection.(im)mutable`
- `Iterable`, `Set`/`Map`/`Seq`(`LinearSeq`/`IndexedSeq`), `View`
- Immutable trait factories:
  - `LinearSeq` and above: `List`, `IndexedSeq`: `Vector`
  - `HashSet`, `HashMap`
- Aliases for top traits and immutable key traits all activated

Implementations

- Immutable: use aliases and factories
- Mutable: use explicitly
  - `ListBuffer`, `ArrayBuffer`, `HashSet`, `HashMap`
  - `Array` gets converted to `WrappedArray`, a mutable `Seq`
- Parallel collections: an external library, will discuss it at the end of the course

Operations

- basic: `add`/`remove`/`size`/`isEmpty`/`find`/`sortWith`/`sum`
- FP-like: `foreach`/`map`/`flatMap`/`filter`, `reduce`/`fold`+`Left`/`Right`, `collect`/`collectFirst`
- adapters: `toArray`/`toList`/`toSeq`/`toStream`/`toIterator`/`to`
- operators: `++`/`++:`/`+:`/`:+`
- recall that on immutable collections, side-effect operations create new collections

1-slide summary of guidelines

Use the proper implementation

- mutable vs immutable collection: use immutable ones unless you have severe performance issues
  - immutable collections generate new objects upon update operations
  - immutable collections can be freely shared, and are easier to work with
  - encapsulate mutabilty! prefer private immutable `var` over private mutable `val`
- kind of collection
  - typical choices: `Map()`, `Set()`, `Seq()`, `List()`, `LazyList()`
  - depending on abstraction/concretion trade-offs
- in local variables/field, you do not need to provide an explicit type

Use the proper type in interfaces

- should better stick to an abstract concept unless implementation details are strictly needed
- use `Iterable`, `Set`, `Seq`, `Map` (occasionally `Linear/IndexedSeq`)
- never expose mutable structures, use views instead

Outline

1. Developing functional data structures
2. Scala collections
3. On (im/)mutability
4. Structuring complex programs
   - Nesting
   - Packages and imports
   - Visibility modifiers

Dealing with (im)mutability: loggers examples

```scala
// a mutable class, using a mutable val
class Logger:
  private val strings = collection.mutable.ListBuffer[String]()
  def log(string: String): Unit = strings.addOne(string)
  def getLog: Iterable[String] = strings.view // a lazy, immutable wrapper

// a mutable class, using an immutable var
class Logger2:
  private var strings = Seq[String]() // List
  def log(string: String): Unit = strings = strings :+ string
  def getLog: Iterable[String] = strings

// an immutable class, using an immutable val
class Logger3 private (private val strings: List[String]):
  def this() = this(List[String]())
  def log(string: String): Logger3 = Logger3(strings :+ string)
  def getLog: Iterable[String] = strings.view // a lazy, immutable wrapper

object TryLogger extends App : // which one is better??
  val logger = new Logger()
  logger.log("str1")
  logger.log("str2")
  logger.log("str3")
  logger.getLog foreach (println(_))

  var logger3 = new Logger3()
  logger3 = logger3.log("str1")
  logger3 = logger3.log("str2")
  logger3 = logger3.log("str3")
  logger3.getLog foreach (println(_))
```

How should we organize mutable concepts of a domain?

Case 1, Mutable class: with a val to mutable data structure

- changes directly affect the inner data structure
- it is essentially the imperative style: what we would do in Java

Case 2, Mutable class: with a var to immutable data structure

- changes affect the class reference to the data structure, which is immutable
- it is a mix of imperative-declarative style: what we often do in Scala

Case 3: Fully immutable class

- methods that change content actually yield a new version of the object
- it follows an FP-idiom, very declarative: FP-style use of Scala, e.g. with State monad

Which one is better?

- choose case 1 only if required by precise performance evaluation, else...
- choose case 2 if the domain concept calls for mutability, else...
- choose case 3 if the domain concept might accept immutability

$\Rightarrow$ namely: "always put side-effects outwards: the core should be declarative!"

Outline

1. Developing functional data structures
2. Scala collections
3. On (im/)mutability
4. Structuring complex programs
   - Nesting
   - Packages and imports
   - Visibility modifiers

Scalable constructs

Scala is scalable: its key constructs can be nested arbitrarily

- `class C{..}`: a template to create objects
- `abstract class C{..}`: a partial template to create objects
- `object O{..}`: an object (singleton/module)
- `trait T{..}`: a mixin (interface)
- `def f(..){..}`: a function (method)
- (`package`s nest into each other, and are very similar to `object`s)

Some useful nesting

- anything inside an object (when extending an `App`)
- classes inside classes (see Java inner classes, but also family polymorphism)
- classes inside objects (see Java nested classes, but also family polymorphism)
- methods inside methods (frequent with recursions)
- anonymous classes inside methods (local helpers)

Scalable constructs

```scala
// nestable constructs
class C
abstract class D
object O
trait T
def f():Unit = {}

println(s"${new C}, $O, ${f()}")
val d = new D(){}   // essentially, anonymous classes
val t = new T{}     // essentially, anonymous classes
println(s"$d, $t, ${new D(){}}, ${new T{}}")

// nestable constructs with some content (in Scala 2 syntax)
class C2 { def a: Int = 1 }
abstract class D2 { def b: Int}
object O2 { def c: Int = 3 }
trait T2 { def d(): Int } // a function interface
def f2():Unit = print("hello")
val d2 = new D2():  // anonymous class
  override def b = 2
val t2: T2 = () => 4    // Java-like fill of functional interface

println(s"${new C2().a}, ${d2.b}, ${O2.c}, ${f2()}, ${t2.d()}")
```

Nesting examples: inner classes

```
object Inner extends App:

  class C(var v: Int):
    class D:
      def g(n: Int): Int =
        n + C.this.v // using outer instance

  val c: C = new C(5) // enclosing instance
  println(new c.D().g(4)) // 9

  object O:
    var v: Int = 10
    class D: // a static inner class
      def g(n: Int): Int =
        n + O.v

  println(new O.D().g(4)) // 14
```

Nesting examples: functions and anonymous classes

```scala
object Anonymous extends App:

  trait Comparator[A]:
    def compare(a: A, b: A): Int

  // a function inside a function
  def min[A](list: List[A], comp: Comparator[A]): Option[A] =
    @annotation.tailrec
    def min2(list: List[A], minSoFar: A): A = list match {
      case h :: t if (comp.compare(h, minSoFar) > 0) => min2(t, minSoFar)
      case h :: t => min2(t, h)
      case _ => minSoFar
    }
    //if (list.isEmpty) None else Some(min2(list.tail, list.head))
    Some(list) filter (! _.isEmpty) map (l => min2(l.tail, l.head))

  // Instantiating an anonymous class
  val comp = new Comparator[Int]():
    override def compare(a: Int, b: Int): Int = a - b

  println(min(List(10,20,5,40), comp)) // Some(5)
  println(min(List(), comp)) // None
  println(min(List(10,20,5,40), _ - _)) // fill of functional interface
```

Packages and importing

Packages in Scala

- fully compatible with Java's packages
- sources should not necessarily stay in a directory matching packages
- packages can define namespaces in source files, as in C#

Flexible importing

- imports can be defined in any point of the code
  - avoid long lists of imports at the beginning, and is more fine-grained
- can import a full package, or single/multiple class, object, trait
- can make a "static import", i.e., importing an object/val's content
- an import can also create an alias of an object/trait/class
  - useful for instance when many packages have the same class
- imports `scala.*`, `java.lang.*` and `scala.Predef.*` are by default
  - `object Predef` includes general helpers, e.g., `println`/`???`
- in Scala 3, we can have top-level methods/vals, defined directly in packages

Packages

```scala
package u05.pack  // the package of all definitions below

val v1 = 1 // when is it initialised?

package a:   // entering a subpackage as a namespace
  object O1
  package b: // entering a further subpackage as a namespace
    def f2() = 1
    object O2
    object A:  // I keep nesting stuff
      object O3
      object B:
        object O4
        class C:
          println(s"$O1,$O2,$O3,$O4,$A,$B,${f()}") // note visibility on top
          println(s"${u05.pack.a.b.A.B.O4}") // full qualification
        def f():Unit =
          println(s"$O1,$O2,$O3,$O4,$A,$B,${new C},$v1,${f2()}")

package a:   // re-entering package u05.pack.a
  object O1B:
    val v = O1 // using u05.pack.a.O1

object Packages extends App:
  println(u05.pack.a.b.A.B.O4)
  println(u05.pack.a.O1B)
```

Importing

```scala
package u05.structure

package p1:
  package p2:
    object O:
      val v = 1

object Import extends App:

  println(u05.structure.p1.p2.O.v) // full qualification
  println(p1.p2.O.v)    // relative qualification

  import p1.*        // importing u05.structure.p1
  println(p2.O.v)

  import p2.*        // the same as: import u05.structure.p1.p2.*
  println(O.v)

  import u05.structure.p1.p2.O.* // static import of O's props
  println(v)

  import u05.structure.p1.p2.{O => TheObject} // alias
  println(TheObject.v)

  import scala.collection.mutable.{ Seq => MutableSeq } // renaming Seq
  println(s"${Seq(10,20,30,40)}, ${MutableSeq(10,20,30,40)}")
```

Understanding packages and imports (1/2)

Packages

- packages are essentially a "visibility" construct, helping encapsulation and deployment of big parts of code
- packages are necessarily a nesting-enabled construct
- each line of code is located into a nested package, and nested objects inside (and other constructs within)
  - package → package → package → ...
  - ... object → object → object → ...
  - ... other constructs (classes, methods, objects)
- being in one such package or object, recursively, means having "imported" its definitions, which are then visible
  - those constructs see definitions on all above objects and packages

Understanding packages and imports (2/2)

Imports

- in each position of code, hence, there are a number of imports available, including:
  - those relative to the packages/objects into which one is located
  - all the explicitly written `import`s declared up to that point
  - default imports `scala.*`, `java.lang.*` and `scala.Predef.*`
- imports allow one to control the set of "identifiers" available
  - methods to call, objects or variables to use, classes to instantiate, types to use
  - can control also the `given`s and `extension`s available—will see what they are
  - with imports one can also locally change names of types (use with care!)
- the use of access modifiers can "bound" availability of identifiers

Access Modifiers

General approach

- `public` is by default, and not a keyword
- a more detailed set of access levels is provided
- note: classes and companion objects mutually share also private definitions

Levels of visibility

- `private`: as in Java
- `protected`: by (this class and) subclasses
- `private`[package]: within *package* (or object)
- `protected`[package]: by subclasses or within *package* (or object)

Access Modifiers

```
package u05.structure

package pa1:
  package pa2:

    class C:
      private object pri             // visible only by C's instances
      private[pa2] object pri_pa2    // visible only up to pa2
      protected object pro           // visible to subclasses
      protected[pa2] object pro_pa2 // visible to subclasses, or up to pa2
      def f(c: C) =
        print(C.g())  // WORKS, can see companion's private stuff

    object C:
      print(new C().pri)          // WORKS, can see class' private stuff
      private def g() = 1

    class D extends C
      //print(pri) // ALL ERRORS

    class E(c: C):
      //print(c.pri,c.pro) // ALL ERRORS
      print(c.pri_pa2, c.pro_pa2) // WORKS!
  end pa2

  class F extends pa2.C:
    //print(pri_pa2) // ALL ERRORS
    print(pro_pa2) // WORKS!

end pa1
```

Access Modifiers: discussion

Levels of visibility

- public a default, private not so frequent..
  - the very frequent pattern "private field and public getters" is often no longer needed due to uniform access
  - very often private methods are used by a single public method, which now can just be nested
  - Java defaulting to package-private is not optimal either
  - instead of many private properties, it is better to build a single private nested abstraction (class or object)
- `private`: as usual, to be used for implementation-only abstractions
- `protected`: when creating inheritance hierarchies
- `private[`*package*`]`: *package* defines the interface/implementation boundary
- `protected[`*package*`]`: *package* defines the interface/implementation boundary

However:

remember that level of accessibility is a key design factor!

Nesting privacy: encapsulating private code

```scala
package u06.structure

class C: // a class with many private properties
  def f(): Unit =
    g1()
    g2()
    g3()
  private def g1(): Unit = { ??? } // local helpers
  private def g2(): Unit = { ??? }
  private def g3(): Unit = { ??? }

class C2: // a refactoring of C, nesting implementation code
  import ImplementationHelpers.* // to facilitate calls

  def f(): Unit =
    g1()
    g2()
    g3()

  private object ImplementationHelpers:
    def g1(): Unit = { ??? }
    def g2(): Unit = { ??? }
    def g3(): Unit = { ??? }
```

A private constructor

```scala
package u06.structure

trait Counter:
  def increment(): Unit
  def getValue(): Int

object Counter:
  def apply(): Counter = new CounterImpl

class CounterImpl private[structure] () extends Counter:
  var count = 0
  override def increment(): Unit = count += 1
  override def getValue(): Int = count

object TryCounters extends App:
  val c = Counter()
  println(c) // u05.structure.CounterImpl@4411d970
```

Wrap-up

Scala is very flexible, you can carefully tune:

- what set of constructions to put into a source
- how to nest constructions
- how to control visibility at each level
- how to define which scope you want to have at the client-side
- mutable/immutable/lazy/strict/OO/FP data structures

And the most advanced things are yet to come..

- full generics, implicits, family polymorphism, monads, DSLs